

EvoMind-DGM: Knowledge-Guided Neuro-Symbolic Evolution for General Intelligence

Yohanes Mesfin
iCog Labs / SingularityNET
yohanesmesfin3@gmail.com

Abstract

Darwin Gödel Machines (DGMs) demonstrate that language-model-based agents can improve their own software scaffolding through archive-based self-modification and empirical selection. However, existing DGM-style systems are primarily oriented toward task-specific improvement, especially in software engineering. General intelligence requires more than improvement on a narrow benchmark: it requires cumulative knowledge acquisition, abstraction, transfer across domains, reasoning under uncertainty, and attention-guided allocation of limited cognitive resources.

We propose EvoMind-DGM, a knowledge-guided neuro-symbolic architecture for open-ended evolution of self-improving agents. EvoMind-DGM extends the DGM framework with a Hyperon-inspired, MeTTa-centered implementation strategy. A DGM evolution controller orchestrates self-improvement around a symbolic runtime built from current Hyperon-family components, especially MeTTa and PeTTa, together with persistent space-backed memory. This memory layer serves as a knowledge substrate for storing agent lineages, task experiences, learned concepts, symbolic policies, failure modes, transfer patterns, and meta-cognitive hypotheses. MOSES is used to evolve reusable symbolic cognitive policies and procedures, including reflection, verification, memory retrieval, task decomposition, and archive-selection strategies. ECAN provides attention-based resource allocation across agents, tasks, memories, policies, and uncertain hypotheses. PLN, including current PeTTa-based implementations, supplies probabilistic reasoning over causality, transfer, risk, and expected improvement. LLMs remain central as semantic generators and implementation-level mutation operators, but are embedded within a broader neuro-symbolic evolutionary system.

The central hypothesis is that self-improving agents guided by persistent symbolic knowledge, probabilistic reasoning, attention allocation, and symbolic program evolution will produce more interpretable, transferable, and generalizable improvements than DGM systems relying primarily on LLM-generated code modifications and fixed benchmark selection. We outline an experimental framework for evaluating EvoMind-DGM on cross-domain adaptation, knowledge reuse, policy transfer, reasoning-guided mutation, and improvement per compute unit.

1 Introduction

Recent progress in large language models (LLMs) has enabled increasingly capable software agents that can reason, write code, call tools, inspect files, and modify their own execution environment. Darwin Gödel Machines (DGMs) extend this direction by allowing agents to iteratively improve themselves through an evolutionary process: a parent agent is selected from an archive, a modified child agent is generated, the child is empirically evaluated, and successful variants are stored for future evolution.

This approach is powerful because it replaces the requirement for formal proof of self-improvement with empirical validation. Rather than proving that a self-modification is beneficial, the system tests whether the resulting agent performs better. This makes DGM-like systems more practical than classical Gödel Machine formulations, particularly when the agent is implemented as a complex software system around a frozen foundation model.

However, the current DGM framing remains limited as a model of general intelligence. Improving benchmark performance in a single domain, such as software engineering, does not by itself imply broad intelligence. A generally intelligent system should not merely become better at one task class. It should accumulate reusable knowledge over time, form abstractions, reason about its own failures, transfer strategies across domains, allocate attention under limited resources, and revise its beliefs based on experience.

We argue that the path from self-improving agents to general intelligence requires shifting the unit of evolution from task-specific agent code to knowledge-bearing cognitive systems capable of accumulating, abstracting, and transferring experience over time.

To address this, we propose **EvoMind-DGM**, a neuro-symbolic extension of Darwin Gödel Machines grounded in a Hyperon-inspired, MeTTa-centered cognitive architecture. EvoMind-DGM combines five core ideas:

1. **DGM-style open-ended evolution:** archive-based self-improvement

of agent variants.

2. **Space-backed knowledge accumulation:** Atomspace-inspired persistent storage of experiences, concepts, policies, lineages, and meta-cognitive hypotheses.
3. **MOSES-based symbolic evolution:** evolutionary discovery of compact, reusable cognitive policies and procedures.
4. **ECAN-based attention allocation:** prioritization of agents, memories, tasks, hypotheses, and evolutionary branches.
5. **PLN-based probabilistic reasoning:** uncertain inference over causality, transfer, risk, and expected improvement.

In this architecture, LLMs remain essential, but they are not treated as the entire intelligence system. Instead, LLMs act as semantic generators, language interfaces, code/policy proposers, and implementation-level mutation engines within a broader neuro-symbolic evolutionary loop coordinated by a DGM controller and symbolic runtime.

The central claim of this paper is that general intelligence requires not only self-modification, but also persistent knowledge, symbolic abstraction, attention, reasoning, and transfer. EvoMind-DGM is proposed as an architecture for studying these mechanisms in a unified, experimentally testable system.

2 Background

2.1 Darwin Gödel Machines

Darwin Gödel Machines are self-improving agent systems inspired by the original Gödel Machine concept but grounded in empirical evolutionary selection. A DGM maintains an archive of agent variants. At each iteration, the system selects a parent agent, generates a modified child agent, evaluates the child on a set of tasks, and stores useful variants in the archive.

The basic DGM loop can be represented as:

$$A_{t+1} = \text{ArchiveUpdate}(A_t, \text{Evaluate}(\text{Mutate}(\text{Select}(A_t)))) \quad (1)$$

where A_t is the archive at time t .

In existing DGM systems, the mutation operator is often implemented using an LLM that modifies the agent’s code, prompts, tools, or workflow.

The evaluation function is typically benchmark-based. This works especially well in software engineering because the ability to modify an agent’s software scaffold is itself closely related to the coding ability being evaluated.

However, this coupling between task performance and self-modification ability is domain-specific. General intelligence requires a broader framework in which improvements are not judged only by performance on one class of tasks, but by their ability to support transfer, abstraction, learning efficiency, and robust adaptation.

2.2 Hyeron, MeTTa, and Space-Based Memory

Hyeron is a neuro-symbolic AGI framework designed to integrate symbolic reasoning, probabilistic inference, evolutionary learning, attention allocation, and neural/LLM-based methods. In current practice, the Hyeron ecosystem is best understood not as a single monolithic runtime containing every cognitive subsystem, but as a family of interoperable components centered on MeTTa runtimes and symbolic spaces. The practical base includes MeTTa implementations such as `hyeron-experimental` and `PeTTa`, together with space APIs and optional backends such as MORK-backed spaces.

In `EvoMind-DGM`, the long-term cognitive memory of the system is best viewed as an `AtomSpace`-inspired, space-backed symbolic memory layer rather than as a single fixed implementation. Unlike a conventional DGM archive, which primarily stores agent variants and benchmark scores, this memory layer stores both evolutionary and cognitive knowledge:

- agent lineages and mutation histories,
- task experiences and outcomes,
- learned concepts and abstractions,
- symbolic cognitive policies,
- transfer patterns across domains,
- failure modes and safety violations,
- probabilistic hypotheses about improvement,
- attention values and resource-allocation priorities.

This turns the archive from a passive record of variants into an active knowledge base over which the system can reason, regardless of whether the underlying implementation uses Hyperon spaces, PeTTa-managed spaces, or other compatible symbolic-memory backends.

2.3 MOSES for Symbolic Evolution

MOSES, or Meta-Optimizing Semantic Evolutionary Search, is an evolutionary program-learning approach that searches over compact symbolic programs or policies. In EvoMind-DGM, MOSES is used not to evolve entire agents at first, but to evolve reusable symbolic cognitive policies.

Examples of policies that MOSES may evolve include:

- when to reflect before answering,
- when to retrieve memory,
- when to call a verifier,
- when to use a tool,
- when to replan,
- when to stop,
- how to aggregate evaluation signals,
- how to decide whether a child agent enters the archive.

This allows EvoMind-DGM to evolve interpretable cognitive control structures rather than relying entirely on opaque LLM-generated modifications. In the current ecosystem, MOSES should be treated as a symbolic subsystem executed through MeTTa-centered workflows, especially repositories such as `metta-moses`, rather than assumed to be a built-in layer of a single Hyperon runtime.

2.4 ECAN for Attention Allocation

ECAN, or Economic Attention Networks, provides a mechanism for allocating limited cognitive resources. In EvoMind-DGM, ECAN is used to assign attention values to agents, tasks, memories, symbolic policies, hypotheses, and evolutionary lineages. In implementation terms, ECAN is best viewed

as a symbolic attention subsystem whose outputs are consumed by the DGM controller.

The need for attention allocation arises because an open-ended DGM-style system can produce a rapidly growing archive of variants and experiences. Evaluating every child agent on every task is computationally infeasible. ECAN provides a principled mechanism for determining where compute and reasoning should be focused.

For example, attention may be allocated to:

- agents with high transfer potential,
- policies that improved multiple domains,
- uncertain hypotheses requiring more evidence,
- underexplored evolutionary lineages,
- tasks likely to reveal generalization failures,
- memories relevant to the current mutation objective,
- safety risks requiring deeper evaluation.

2.5 PLN for Probabilistic Reasoning

PLN, or Probabilistic Logic Networks, provides uncertain reasoning over symbolic knowledge. In EvoMind-DGM, PLN reasons over symbolic-memory representations of agents, tasks, policies, failures, and improvements. In the current stack, PLN is naturally treated as a PeTTa/MeTTa reasoning subsystem invoked by the controller rather than assumed to be a built-in service of the base Hyperon runtime.

PLN is used to infer:

- whether a mutation likely caused an observed improvement,
- whether a policy is likely to transfer across domains,
- whether a performance gain is likely benchmark-specific,
- whether a child agent introduces safety risk,
- which task would best reduce uncertainty,
- which evolutionary branch has the highest expected value.

In this way, EvoMind-DGM does not merely record which agents performed better. It develops probabilistic beliefs about why they improved and whether those improvements are generally useful.

3 Motivation: From Task-Specific Improvement to General Intelligence

A key limitation of current DGM-style systems is that they are commonly evaluated through task-specific benchmarks. While benchmark improvement is necessary, it is not sufficient for general intelligence.

We define general intelligence in this context as:

The ability to acquire, represent, abstract, transfer, and apply knowledge across changing tasks and domains under limited resources.

This definition emphasizes five capabilities:

1. **Acquisition:** learning from experience.
2. **Representation:** storing knowledge in reusable form.
3. **Abstraction:** extracting general patterns from specific experiences.
4. **Transfer:** applying knowledge across domains.
5. **Resource allocation:** focusing limited compute and attention effectively.

A DGM system that only stores agent code and benchmark scores lacks the structure needed to support these capabilities. It may discover useful modifications, but it does not necessarily convert experience into reusable knowledge.

EvoMind-DGM addresses this by introducing a persistent space-backed knowledge substrate and by using MOSES, ECAN, and PLN to evolve, prioritize, and reason over knowledge-bearing cognitive structures.

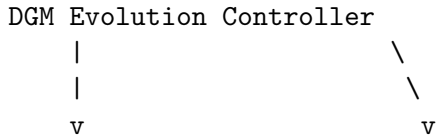
4 EvoMind-DGM Architecture

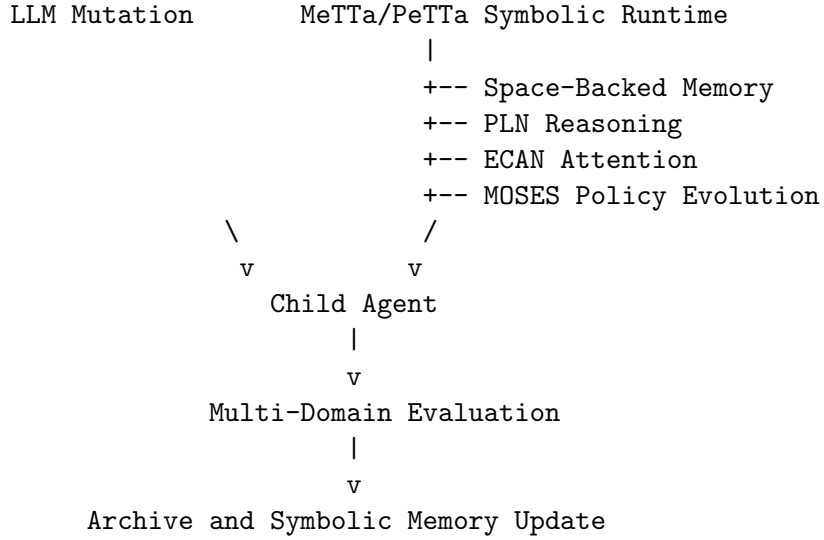
4.1 Overview

EvoMind-DGM is a neuro-symbolic architecture for open-ended evolution of self-improving agents. Conceptually, it still combines DGM-style evolution, persistent symbolic memory, symbolic policy evolution, attention allocation, probabilistic reasoning, and LLM-based mutation. Implementation-wise, however, these are treated as cooperating subsystems coordinated by a DGM evolution controller rather than as one already-integrated monolithic runtime. Its high-level loop is:

1. A parent agent or policy is selected from the archive.
2. Relevant prior knowledge is retrieved from the space-backed memory layer.
3. The symbolic runtime invokes PLN to reason about likely causes, risks, and transfer opportunities.
4. The symbolic runtime invokes ECAN to allocate attention to promising agents, memories, tasks, and hypotheses.
5. The symbolic runtime invokes MOSES to evolve symbolic cognitive policies or procedures.
6. An LLM proposes implementation-level modifications or task-specific strategies.
7. A child agent is constructed.
8. The child agent is evaluated across multiple domains.
9. Results, traces, and extracted abstractions are stored in the space-backed memory layer.
10. Archive entries, attention values, and probabilistic beliefs are updated.

The architecture can be summarized as:





4.2 Core Components

4.2.1 Agent Archive

The archive stores agent variants, symbolic policies, lineages, evaluation histories, and metadata. Each archive entry may include:

```

AgentVariant {
  id
  parent_id
  code_or_configuration
  symbolic_policy
  task_scores
  transfer_scores
  safety_scores
  attention_value
  mutation_description
  causal_hypotheses
  evaluation_logs
}

```

Unlike the original DGM archive, EvoMind-DGM's archive is linked to the symbolic memory layer, allowing agent histories to become part of the system's long-term knowledge.

4.2.2 Space-Backed Knowledge Substrate

The knowledge substrate stores structured knowledge extracted from agent experiences. In a current implementation, this layer may be realized using Hyperon spaces, PeTTa-managed MeTTa spaces, or MORK-backed spaces while remaining conceptually Atomspace-inspired. This includes:

- episodic knowledge: what happened during a task,
- semantic knowledge: general concepts and task structures,
- procedural knowledge: reusable policies and workflows,
- meta-cognitive knowledge: knowledge about the system’s own reasoning,
- goal/value knowledge: constraints and optimization objectives.

An example representation may be:

```
(AgentVariant Agent-17)
(ParentOf Agent-9 Agent-17)
(UsesPolicy Agent-17 ReflectBeforeToolUse)
(Improves Agent-17 ResearchComprehension)
(TransfersTo ReflectBeforeToolUse CodingTask)
(TruthValue (TransfersTo ReflectBeforeToolUse CodingTask) 0.72 0.61)
```

4.2.3 MOSES Symbolic Evolution Layer

MOSES searches over symbolic policies represented in a constrained grammar. A policy may take features of the task, agent state, memory state, and uncertainty estimates as input, and output a cognitive action. In implementation terms, MOSES candidates may be generated and evaluated inside a MeTTa-centered symbolic runtime and then exposed to the controller as reusable policies.

Example policy schema:

```
IF uncertainty > theta_1 AND task_difficulty = high
THEN call_verifier

IF memory_relevance > theta_2 AND task_type = research
THEN retrieve_memory

IF progress < theta_3 AND tool_failures > 1
THEN replan
```

MOSES evolves such policies based on multi-objective fitness, including task success, transfer, interpretability, safety, and compute cost.

4.2.4 ECAN Attention Layer

ECAN assigns attention values to entities in the symbolic memory layer and the archive. A simplified attention score may be:

$$Att(x) = \alpha I(x) + \beta N(x) + \gamma U(x) + \delta T(x) + \eta S(x) - \lambda C(x) \quad (2)$$

where:

- $I(x)$ is recent improvement potential,
- $N(x)$ is novelty,
- $U(x)$ is uncertainty,
- $T(x)$ is transfer potential,
- $S(x)$ is safety relevance,
- $C(x)$ is evaluation cost.

ECAN guides parent selection, task selection, memory retrieval, evaluation depth, and safety prioritization. In a practical implementation, its scores are made available to the DGM controller as part of the symbolic runtime state.

4.2.5 PLN Reasoning Layer

PLN reasons over symbolic-memory knowledge to estimate causal and transfer relationships. In the current ecosystem, this layer can be naturally implemented in PeTTa/MeTTa and queried by the DGM controller during mutation and evaluation planning.

Example inference targets include:

Policy P improves task family F.
 Policy P transfers from domain D1 to domain D2.
 Mutation M caused improvement I.
 Agent A has high risk under task type T.
 Task T will reduce uncertainty about hypothesis H.

PLN truth values can be used by ECAN and MOSES. For instance, if PLN infers that a policy has high transfer potential but low confidence, ECAN may allocate more attention to testing that policy.

4.2.6 LLM Mutation Layer

LLMs provide semantic flexibility. They may:

- generate code modifications,
- translate symbolic policies into executable agent behavior,
- summarize task failures,
- propose new tasks,
- explain mutation rationales,
- generate hypotheses for PLN to evaluate,
- convert natural-language experience into symbolic-memory structures.

The LLM is therefore not the whole cognitive system. It is a generative and interpretive component within a larger neuro-symbolic loop.

5 Knowledge-Guided Self-Improvement

The key architectural difference between DGM and EvoMind-DGM is the role of knowledge.

In standard DGM, the loop is:

$$\text{Agent} \rightarrow \text{Mutation} \rightarrow \text{Evaluation} \rightarrow \text{Archive} \quad (3)$$

In EvoMind-DGM, the loop becomes:

$$\text{Experience} \rightarrow \text{Knowledge} \rightarrow \text{Reasoning} \rightarrow \text{Attention} \rightarrow \text{Evolution} \rightarrow \text{New Experience} \quad (4)$$

This allows the system to accumulate knowledge about its own learning process.

5.1 Knowledge Extraction

After each evaluation, the system extracts knowledge from the child agent’s behavior. This may include:

- task success or failure,
- tools used,
- reasoning steps,
- memory retrieval quality,
- verifier usage,
- failure causes,
- policy decisions,
- safety events,
- compute cost,
- cross-domain effects.

This information is converted into Atomspace representations.

5.2 Abstraction and Transfer

The system should not merely memorize task outcomes. It should abstract reusable patterns.

For example, from multiple task experiences, it may infer:

`Reflect-before-tool-use` improves performance when uncertainty is high.
`Evidence mapping` improves research-comprehension tasks.
`Verifier calls` improve math accuracy but increase cost.
`Memory retrieval` helps long-context tasks but can harm short tasks.

These abstractions can then guide future mutations and evaluations.

6 Research Questions

This work investigates the following research questions:

1. **RQ1:** Can a DGM-style self-improving agent accumulate reusable knowledge in Atomspace that improves future learning and adaptation?
2. **RQ2:** Can MOSES-evolved symbolic cognitive policies transfer across task domains better than purely LLM-generated prompt or code modifications?
3. **RQ3:** Can ECAN-based attention allocation reduce evaluation cost while preserving or improving open-ended exploration?
4. **RQ4:** Can PLN-based reasoning over agent histories improve mutation selection, task selection, and safety assessment?
5. **RQ5:** Does EvoMind-DGM produce more interpretable and generalizable improvements than DGM-only baselines?

7 Experimental Design

7.1 Systems Compared

We propose comparing four systems:

1. **Baseline LLM Agent:** fixed workflow, no self-improvement.
2. **DGM-only Agent:** archive-based self-modification using LLM-generated mutations.
3. **DGM + MOSES + ECAN:** symbolic policy evolution and attention-guided resource allocation.
4. **EvoMind-DGM:** full knowledge-guided architecture with Atomspace, MOSES, ECAN, PLN, and LLM mutation.

7.2 Task Domains

To evaluate generality, tasks should span multiple domains:

- software engineering and code repair,
- mathematical or logical reasoning,

- research-paper comprehension,
- tool-use and planning tasks,
- memory-dependent tasks,
- safety and robustness tests.

The goal is not merely to maximize performance in each domain independently, but to test whether improvements discovered in one domain transfer to others.

7.3 Evolvable Objects

EvoMind-DGM may evolve:

- reflection policies,
- verifier-selection policies,
- memory-retrieval policies,
- task-decomposition procedures,
- tool-use policies,
- archive-selection rules,
- fitness aggregation functions,
- task-selection policies,
- safety-check policies.

7.4 Metrics

Evaluation should include both task performance and general-intelligence-oriented metrics.

Metric	Description
Task success	Performance on individual task domains
Transfer score	Improvement transferred across domains
Knowledge reuse	Frequency and utility of retrieved Atomspace knowledge
Policy reuse	Number of domains benefiting from the same symbolic policy
Improvement per compute	Performance gain per LLM call or evaluation cost
Archive diversity	Diversity of retained agent/policy variants
Interpretability	Human readability of evolved policies and hypotheses
Safety stability	Absence of safety regressions during self-improvement
Adaptation speed	Ability to improve on new task families with fewer examples

Table 1: Proposed evaluation metrics for EvoMind-DGM.

8 Expected Contributions

This paper makes the following expected contributions:

1. A conceptual architecture extending Darwin Gödel Machines from task-specific agent improvement toward knowledge-guided general intelligence.
2. A proposal for using Atomspace as a persistent knowledge substrate for self-improving agents.
3. A mechanism for evolving reusable symbolic cognitive policies using MOSES.
4. An ECAN-based approach to attention and resource allocation in open-ended agent evolution.
5. A PLN-based reasoning layer for inferring causality, transfer, risk, and expected improvement.
6. An experimental framework for evaluating generality through knowledge reuse, policy transfer, and adaptation efficiency.

9 Safety Considerations

Self-improving agents introduce significant safety concerns. EvoMind-DGM addresses these concerns by making safety part of the architecture rather than treating it as an external filter.

Safety-relevant knowledge is stored in Atomspace, including:

- unsafe mutations,
- tool misuse events,
- hallucination patterns,
- failed verification cases,
- policies associated with risk,
- constraints on self-modification.

MOSES can evolve explicit safety policies. ECAN can allocate attention to risky or uncertain branches. PLN can reason about the probability that a mutation creates a safety regression. Archive selection can reject or quarantine agents that improve capability while reducing safety.

A simplified archive decision rule may be:

```
IF safety_regression(agent) > threshold:
    reject_or_quarantine(agent)
ELSE IF transfer_gain(agent) > 0 AND capability_gain(agent) > 0:
    archive(agent)
ELSE IF novelty(agent) high AND risk(agent) low:
    archive_as_stepping_stone(agent)
```

10 Limitations

Several limitations remain.

First, implementing a full EvoMind-DGM system requires orchestration across an external DGM controller, LLM agents, MeTTa/PeTTa runtimes, and space implementations rather than simply enabling a single integrated Hyperon package. Second, evaluating general intelligence remains difficult, and benchmark design must avoid narrow overfitting. Third, translating unstructured LLM traces into reliable symbolic-memory representations is non-trivial. Fourth, MOSES-evolved policies may be interpretable but still difficult to validate across open-ended environments. Fifth, ECAN and PLN must be adapted to operate effectively over large and evolving agent archives. Sixth, version skew and interface mismatch across Hyperon-family repositories may introduce substantial engineering overhead.

This paper therefore presents EvoMind-DGM as a research architecture and experimental agenda rather than a completed AGI system.

11 Conclusion

Darwin Gödel Machines provide a powerful framework for open-ended self-improvement, but task-specific benchmark improvement alone is insufficient for general intelligence. General intelligence requires systems that accumulate knowledge, abstract from experience, transfer strategies across domains, reason under uncertainty, and allocate attention under limited resources.

EvoMind-DGM extends DGM with a Hyperon-inspired neuro-symbolic architecture centered on a DGM controller, a MeTTa/PeTTa symbolic runtime, and persistent space-backed memory. Within this architecture, MOSES evolves reusable symbolic cognition, ECAN allocates attention and compute, PLN reasons over uncertainty and transfer, and LLMs provide semantic and implementation-level generation. Together, these components define a path from self-improving agents toward knowledge-guided general intelligence.

The central research challenge is to determine whether such systems can acquire reusable cognitive knowledge over time and use it to improve adaptation across domains. If successful, EvoMind-DGM would represent a step toward open-ended, interpretable, and knowledge-bearing self-improving intelligence.